

Flask POST Request Practice

20 detailed beginner exercises • Questions 11–30 • JSON • Validation • REST basics

How to use this workbook

Solve each question yourself before looking at documentation. Use Postman, curl, or another API client to send requests. Keep the data in Python lists for now—no database is required.

Recommended progression: Basic POST → JSON → validation → lists → IDs → status codes → headers → nested JSON → mini API.

Level 1 — Basic POST APIs

11. Employee Registration API

Create a Flask endpoint **POST /employees** that registers a new employee.

Request JSON:

```
{ "name": "Rahul", "department": "IT", "salary": 45000 }
```

Your task: Read the JSON body, extract all three fields, validate that none is missing, and return the employee details as JSON.

Expected learning: POST routes, `request.get_json()`, JSON responses, and basic validation.

12. Student Registration API

Create **POST /students** to register a student.

Accept name, age, and course.

If all values are present, return a message such as `Student registered successfully` along with the submitted data.

If any field is missing, return an error response with HTTP **400**.

Expected learning: Checking request data and choosing an appropriate status code.

13. Contact Form API

Create **POST /contact** for a simple contact form.

Accept name, email, and message in JSON.

If valid, return `Message received successfully` and the submitted contact information.

If message is empty or missing, return HTTP **400**.

Expected learning: Handling strings, missing fields, and simple business rules.

14. Feedback API with Rating Validation

Create **POST /feedback**.

Accept name, rating, and comment.

The rating must be an integer from **1 to 5**.

Return **201** when feedback is accepted. Return **400** when the rating is outside the allowed range or required data is missing.

Expected learning: Validating numeric input and returning different HTTP status codes.

15. Simple Login API

Create **POST /login**.

Accept username and password as JSON.

For this exercise only, consider admin / 1234 as valid credentials.

Return **200** with Login successful for valid credentials. Otherwise return **401** with an error message.

Note: This is only a learning exercise; do not store real passwords like this in production.

Expected learning: Reading POST data and implementing a simple condition.

Level 2 — JSON & Validation

16. User Age Validation

Create **POST /users** that accepts name and age.

The user must be at least **18 years old**.

Example valid request: { "name": "Puneeth", "age": 27 }.

If age is below 18, return HTTP **400** and clearly explain the validation error.

Also handle missing or invalid age values.

Expected learning: Type conversion, comparisons, and validation.

17. Email Validation

Create **POST /register** that accepts name and email.

For this beginner exercise, consider an email valid if it contains @ and a dot after the @.

Reject invalid emails with HTTP **400** and a useful message.

Expected learning: String validation and conditional logic. You do not need a full email-validation library.

18. Password Length Validation

Create **POST /register** with username and password.

The password must contain at least **8 characters**.

If it is shorter, return HTTP **400** and explain the requirement.

If it passes, return a success response. Do not return the password in the response.

Expected learning: Length validation and safe response design.

19. Return Multiple Validation Errors

Create **POST /users** accepting name, email, and age.

Instead of stopping at the first error, collect every validation error and return them together.

Example response:

```
{ "errors": ["name is required", "email is required", "age must be 18 or older"] }
```

Return **400** if one or more errors exist; otherwise return success.

Expected learning: Building an error list and performing multiple validations in one request.

20. Accept JSON Only

Create **POST /users** that accepts JSON requests only.

If the client sends a JSON body, process it normally.

If the request is sent as form data or without JSON, return HTTP **400** with a clear message such as JSON body is required.

Test the endpoint using Postman or another API client by changing the Content-Type.

Expected learning: Content type, JSON request handling, and API testing.

Level 3 — Working with Python Lists

21. Create a Task

Create **POST /tasks**.

Accept a JSON body containing a `title`, for example `{ "title": "Learn Flask" }`.

Maintain a Python list called `tasks` in memory.

When a valid request arrives, add the task to the list and return the newly created task.

Do not use a database yet.

Expected learning: Connecting request data with Python data structures.

22. Create Products with Automatic IDs

Create **POST /products**.

Accept `name` and `price`.

Store products in a Python list and automatically generate an integer `id` for every product, such as 1, 2, 3...

Return the complete created product with HTTP **201**.

Expected learning: Maintaining application state and generating IDs.

23. Prevent Duplicate Email

Create **POST /users** and maintain users in a Python list.

Each user contains `id`, `name`, and `email`.

Before adding a new user, check whether another user already has the same email.

If it exists, return HTTP **409 Conflict**. Otherwise create the user and return **201**.

Expected learning: Searching a list and understanding the purpose of HTTP 409.

24. Order Creation and Total Price

Create **POST /orders**.

Accept `product`, `price`, and `quantity`.

Calculate `total = price × quantity`.

Return the order details plus the calculated total.

Reject zero or negative price/quantity values with HTTP **400**.

Expected learning: Calculating values from request data and validating numeric input.

25. Todo Creation with Default Value

Create **POST /todos**.

Accept `title` and optional `description`.

When a todo is created, automatically add `completed: false` even if the client does not send it.

Generate an ID and store the todo in a list.

Return the created todo with HTTP **201**.

Expected learning: Optional fields, default values, IDs, and object creation.

Level 4 — REST Concepts & Mini Challenge

26. Use the Correct HTTP Status Codes

Improve a **POST /users** endpoint so it uses meaningful status codes.

Use **201 Created** when a user is successfully created, **400 Bad Request** for invalid/missing input, and **409 Conflict** when the email already exists.

Return a JSON body explaining the result in every case.

Expected learning: Designing predictable REST API responses.

27. Return a Location Header

Create **POST /users** that generates a user ID and returns the newly created user.

After creation, also return a Location response header containing the URL of the new resource, for example `/users/5`.

Return HTTP **201**.

Expected learning: HTTP response headers and the REST concept of identifying a newly created resource.

28. Handle Nested JSON

Create **POST /employees** accepting nested JSON:

```
{ "name": "Puneeth", "contact": { "email": "test@example.com", "phone": "9999999999" } }
```

Extract `name`, `contact.email`, and `contact.phone`.

Validate that the contact object and its required fields exist.

Expected learning: Navigating nested dictionaries safely.

29. Validate Boolean Values

Create **POST /users** accepting `name` and `is_active`.

The `is_active` value must be a real JSON boolean: `true` or `false`.

Do not accept strings such as `"true"` as valid boolean input for this exercise.

Return HTTP **400** when the value is invalid.

Expected learning: Understanding JSON data types and Python type checking.

30. ■ Mini Project — Student Management API

Build a small in-memory **Student Management API** using **POST /students**.

Request: `{ "name": "Rahul", "age": 21, "course": "Python", "email": "rahul@example.com" }`

Requirements: validate required fields; validate age; validate email; generate a unique integer ID; prevent duplicate emails; store students in a Python list; return **201** for successful creation; return **400** for invalid input; return **409** for duplicate email; and return the created student as JSON.

Bonus: Add `GET /students` to see all students and `GET /students/<id>` to retrieve one student.

Expected learning: Combine everything from Questions 11–29 into one small REST API.

Quick Revision Checklist

Before moving beyond beginner POST requests, make sure you can explain and use each item below.

- ✓ Create a Flask route that accepts POST requests.
- ✓ Read JSON using `request.get_json()`.
- ✓ Understand the difference between JSON data and form data.
- ✓ Check whether required fields are missing.
- ✓ Validate strings, numbers, booleans, and simple business rules.
- ✓ Return JSON responses from Flask.
- ✓ Use HTTP 200, 201, 400, 401, and 409 appropriately.
- ✓ Store temporary data in a Python list.
- ✓ Generate simple IDs.
- ✓ Detect duplicate records.
- ✓ Return response headers such as Location.
- ✓ Read nested JSON objects safely.
- ✓ Test POST APIs using Postman or curl.

Next step after completing these: GET requests → PUT/PATCH → DELETE → CRUD API → database integration with SQLAlchemy.